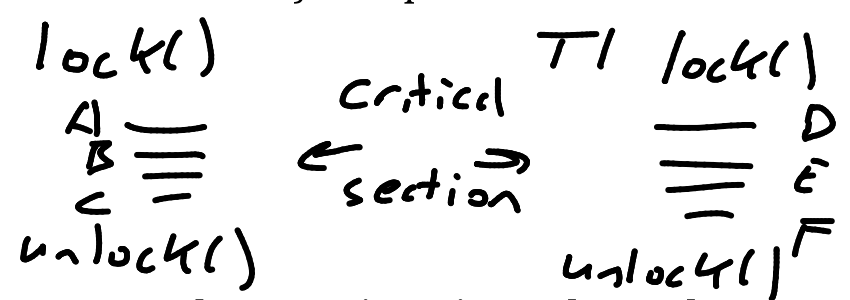


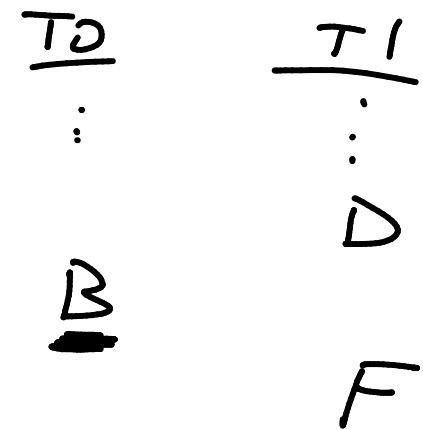
Look at Algorithm 1 on the locking worksheet.

Here is a proof of mutual exclusion, by contradiction.



Suppose the lock doesn't satisfy mutual exclusion. Then we can have a situation where the two critical sections interleave. So there's some piece of code in the critical section from thread 0 that's surrounded on both sides by code from thread 1, or the other way around. (Otherwise, we've there's no interleaving.)

Let's assume it's thread 0 that's got code surrounded by thread 1.



Why can we assume that in this proof? It might not be true.

do the proof all over again the other way 'round, if you like

Then, let's look at that single line of code in thread 0 that's surrounded by code from thread 1. At that point in time...

From thread 1's perspective: flag[1] must be true.

Why?

T1 is past lock and it set $flag[1] = true$ in lock code (and no one else is changing it)

From thread 0's perspective: flag[1] must have been false at some point in the past before this line of code.

Why?

T0 got past lock() - got out of while loop, ($flag[1]$ became false)

$B \Rightarrow C$

1. So we can say "T0 read that $flag[1]$ was false before T1 set $flag[1]$ to true."

Why? Because T0 must have read false,

then T1 must have it to true afterwards, because otherwise T0 would still be stuck.

$A \Rightarrow B$ 2. T0 can also observe that it set $flag[0]$ to be true before T0 read that $flag[1]$ was false.

Why? T0 broke out of while loop, so it did all steps

Combine the above two. 2 followed by 1 lets us conclude that

$A \Rightarrow C$

• "T0 set flag[0] to true before T1 set flag[1] to true."

~~(*)~~

How did we conclude this?

(2) + (1) = this step

(A)

Let's go a step further: T1 sets flag[1] to true one line before it reads that flag[0] is false, so we can then conclude:

(B)

(✓)

"T0 set flag[0] to be true before T1 reads flag[0] to be false."

How did we conclude this?

Start with ~~(*)~~, after (A), then (B) happens (from code)

Postpone the above for now.

and it read false because it got through the while.

Now, let's then start all over and do it all again, but reverse the roles of the threads. So just pretend we flipped everything, and that we assumed we were looking at a line of code in T1's critical section that had code surrounded by code from T0. Then we derive

➔ 1. "T1 read that flag[0] was false before T0 set flag[0] to true."

Where does this come from?

(1) from previous page, flip all the roles.

Which leads us to our contradiction, when comparing 1 and 1.

(✓)

How is this a contradiction?

this can't exist with (✓)